K8S-09: Troubleshooting Kubernetes Monitoring

> **Series:** K8S — Kubernetes Monitoring | **Notebook:** 9 of 14 | **Created:** January 2026 | **Last Updated:** 11/03/2026

Debugging Dynatrace Monitoring in Kubernetes

When monitoring doesn't work as expected, systematic troubleshooting is essential. This notebook covers common issues, diagnostic procedures, and resolution steps for Dynatrace Kubernetes monitoring.

Table of Contents

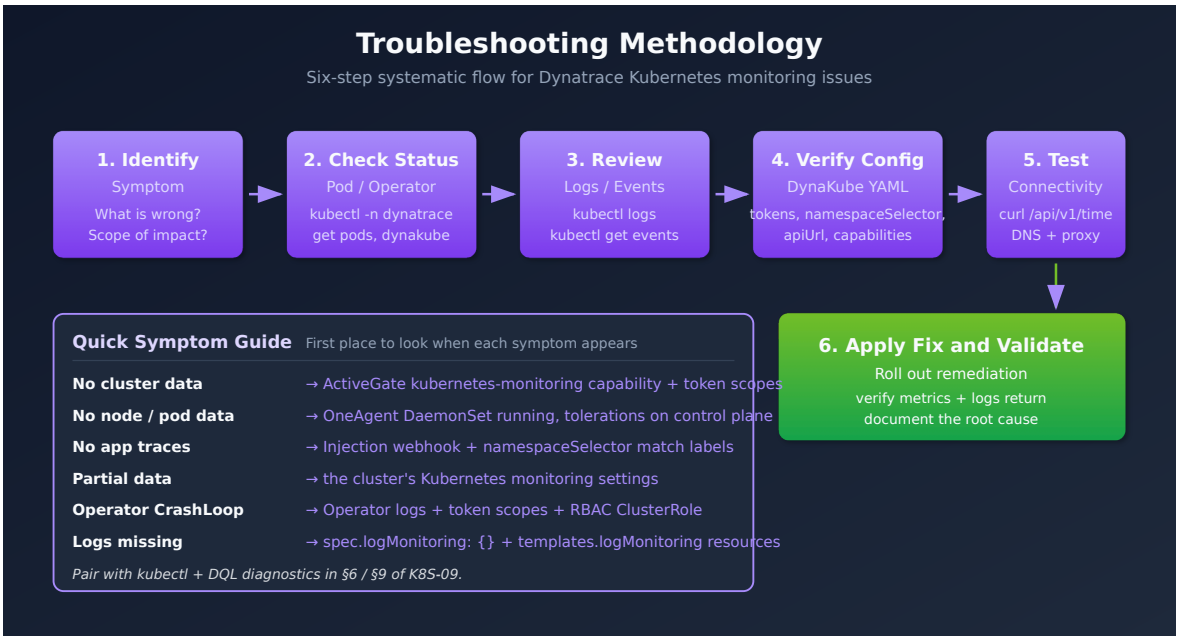
1. Troubleshooting Methodology
2. Operator Issues
3. OneAgent Issues
4. ActiveGate Issues
5. Injection Issues
6. Data Collection Issues
7. Connectivity Issues
8. Diagnostic Commands
9. DQL-Based Diagnostics
10. Symptom-to-Resolution Index

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS/Managed access
kubectl	Configured for your cluster
Permissions	Cluster admin or debug access
Knowledge	K8S-01 through K8S-03

1. Troubleshooting Methodology

Systematic Approach



Common Symptom Categories

Symptom	Likely Component	Start Here
No cluster data	ActiveGate	Section 4
No node/pod data	OneAgent	Section 3
No app traces	Injection	Section 5

Symptom	Likely Component	Start Here
Partial data	Data collection	Section 6
Operator errors	Operator	Section 2

First Checks

```
# DynaKube status
kubectl -n dynatrace get dynakube

# All Dynatrace pods
kubectl -n dynatrace get pods

# Recent events
kubectl -n dynatrace get events --sort-by='.lastTimestamp' | tail -20
```

2. Operator Issues

Operator Not Running

Symptoms:
- DynaKube CR shows no status
- OneAgent pods not created

Diagnostic:

```
# Check operator deployment
kubectl -n dynatrace get deployment dynatrace-operator

# Check operator logs
kubectl -n dynatrace logs -l app.kubernetes.io/name=dynatrace-operator --
tail=100
```

Common Causes:

Cause	Resolution
Missing CRDs	Reinstall operator with Helm
RBAC issues	Check ServiceAccount permissions

Cause	Resolution
Resource limits	Increase operator resources
Webhook failure	Check webhook certificates

DynaKube Stuck in Deploying

Diagnostic:

```
# Describe DynaKube
kubectl -n dynatrace describe dynakube dynakube

# Check conditions
kubectl -n dynatrace get dynakube -o jsonpath='{.items[*].status.conditions}'
| jq .
```

Common Causes:

Cause	Resolution
Invalid API token	Regenerate and update secret
Network policy blocking	Allow egress to Dynatrace
Image pull failure	Check pull secrets

Known Issues by Operator Version

Some failures are not configuration mistakes — they are known defects in specific Dynatrace Operator versions. Before deep-diving a symptom, check whether your Operator version falls in an affected window:

First affected	Fixed / Mitigated	Issue
Not stated in the release notes	**1.9.1** and **1.10.3** (both released October 15, 2026 — adopt on your own schedule), and **1.11.0**	**CSI driver cleanup removes code modules still in use.** The Operator 1.10.3 release notes (DT docs) say: *"Fixed an issue where the CSI driver cleanup could delete OneAgent code module binaries still in use by running pods. Without this fix, a container restart (for example, after an out-of-memory termination) could leave the application without its OneAgent files, so the application would no longer be monitored."* The 1.9.1 (DT docs) and 1.11.0 (DT docs) release notes carry the same fix.
1.9.0	**Still open at 1.11.0** — listed under *Known issues* in the 1.11.0 release notes; the fix is cluster-side	**OpenShift only** — the injected init container carries `seccompProfile: RuntimeDefault` by default from 1.9.0 (the `feature.dynatrace.com/init-container-seccomp-profile` flag flipped to `true`). Pods that previously ran under the **`anyuid`** SCC are rejected at admission with `Forbidden: seccomp may not be set` — before scheduling, with no Dynatrace component logging an error. This is not a defect being fixed in a later Operator: the default is unchanged through 1.10.2 (verified from the operator source at each tag, 08/27/2026), the 1.11.0 release notes still list it (*"Since Dynatrace Operator 1.9.0, a RuntimeDefault seccomp profile is applied to the Dynatrace init container by default."*), and the flag that disables it was **marked deprecated for removal** at 1.10.2. The durable fix is a custom SCC that permits `runtime/default` seccomp for the affected workloads. **FAQ-13** covers admission mechanics, the SCC matrix, the custom-SCC manifest and the RBAC gotcha.
1.10.1 and earlier	**1.10.2** (released July 30, 2026 — staged adoption; upgrade on your own schedule)	Four defects fixed at once: (1) **Kubernetes workload and namespace tagging regression** — where several rules matched the same key, each later rule overwrote the previous one; from 1.10.2 **only the first matching rule for a key applies**, which is a *behaviour change*, not just a fix (see K8S-10). (2) **`dynatrace-webhook`** **`CrashLoopBackOff`** **on gVisor** runtime-class nodes. (3) **Injected pods hanging at startup** when the OneAgent-binary download init container timed out — the timeout is now 15 minutes. (4) **Metadata-enrichment rules that cannot be applied are now logged** instead of being silently disregarded, so a rule that never took effect is finally visible.

First affected	Fixed / Mitigated	Issue
1.10.1 (OpenShift manifests)	1.10.1 (corrected manifests re-uploaded 07/2026)	**OpenShift only** — on a fresh install, or an upgrade where the operator resource was deleted first, the operator pod hangs at `Init:0/1` and `dynatrace-webhook` pods stay `0/1`, waiting for a `dynatrace-webhook-certs` secret that is never created. Root cause is a bootstrap deadlock between the two Operator 1.10.x init containers: **`crd-storage-migrator`** waits for webhook readiness, the webhook waits for its cert secret, and the OpenShift manifest omitted the **`webhook-cert-generator`** init container that creates it. Vanilla-Kubernetes Helm and kubectl manifests ship it and are unaffected. **Fix:** re-download the corrected v1.10.1 OpenShift manifests and reinstall — there is no new Operator version. Interim: run the operator image's `certgen` command as a `Job` or `oc run` pod to create the secret and patch the webhook caBundle.
1.10.0	1.10.1	**Auto-update failures** across ActiveGate, CodeModule, and OneAgent, plus the **OneAgent rollout integrity check hanging** — a rollout that never completes rather than one that visibly fails. The 1.10.0 release notes themselves recommend skipping the release, and it is now flagged **`prerelease: true`** on the GitHub releases page — a structural signal you can check yourself before pinning. **Fix:** upgrade to 1.10.1.
1.10.0	1.10.1 (via feature flag)	**`classicFullstack` only** — OneAgent fails TLS certificate verification when connecting through an **in-cluster ActiveGate**. **Fix:** upgrade to 1.10.1, which introduces the `feature.dynatrace.com/automatic-tls-certificate` feature flag on the DynaKube to restore automatic certificate handling. Do **not** confuse this with the 1.1.0 → 1.5.1 row below — see the note after the table.
1.8.0	1.8.1	OpenShift OperatorHub-based install — Kubernetes API monitoring missing (1.8.1 shipped as a hotfix ten days after 1.8.0)
1.3.0	1.4.1 (mitigated)	CSI driver pods crash frequently — liveness-probe failures under high simultaneous mount load (see below)
1.1.0	1.5.1	OneAgent pods unable to validate the SSL certificate of a containerized ActiveGate after an Operator upgrade

```
# Which Operator version am I running?
kubectl -n dynatrace get deployment dynatrace-operator \
  -o jsonpath='{.spec.template.spec.containers[0].image}'

# Which init containers does the operator pod have? (maps an Init:0/1 hang to
a diagnosis)
kubectl -n dynatrace get pod -l app.kubernetes.io/name=dynatrace-operator \
  -o jsonpath='{.items[*].spec.initContainers[*].name}'
```

Two different ActiveGate-certificate failures — pick the right row. The table has two entries whose *symptom* is identical ("OneAgent cannot validate the certificate of a containerized ActiveGate"), and routing to the wrong one wastes a triage cycle:

You are on	Deployment mode	Row that applies	Fix
1.1.0 – 1.5.0	any	1.1.0 → 1.5.1	Upgrade to 1.5.1 or later
1.10.0	`classicFullstack` with an in-cluster ActiveGate	1.10.0 → 1.10.1	Upgrade to 1.10.1 and set `feature.dynatrace.com/automatic-tls-certificate`

A reader on 1.10.x who matches on symptom alone lands on the 1.5.1 fix, which does not cover them — they are already far past 1.5.1. Check your Operator version *and* your `spec.oneAgent` mode before acting.

Tip: The Kubernetes/OpenShift troubleshooting map (Dynatrace community) maintains this known-issues list as new Operator versions ship — check it alongside the Operator releases (Dynatrace GitHub) when triaging a failure that appeared right after an upgrade.

OpenShift `Init:0/1` on 1.10.1: a fresh-install or operator-deleted-upgrade hang where the operator sits at `Init:0/1` and the webhook waits for its cert secret is the 1.10.1 manifest defect in the table above — **re-pull the OpenShift manifests before any deeper triage.** Details and the `certgen` workaround: Heads-up: Unable to deploy Operator 1.10.1 on fresh clusters (Dynatrace community — requires sign-in).

CSI Driver Crash Loops Under High Mount Load

On Operator 1.3.0+, clusters with a large number of simultaneous volume mounts per node (typically high pod density) can drive the CSI driver `server` container into a liveness-probe crash loop. Symptoms: high system load, high kubelet CPU, pods waiting on volume provisioning, `FailedMount` events referencing a missing `csi.sock`, and the CSI driver pod restarting with back-off.

Mitigations, in order of preference:

1. **Upgrade the Operator to 1.4.1 or later** — the liveness-probe parameters were adjusted to tolerate mount-storm load.
2. **Raise (or remove) the CSI `server` container resource limits** via Helm values:

```
csidriver:
  server:
    resources:
      requests:
        cpu: 250m
        memory: 200Mi
      limits:
        cpu: 1000m
        memory: 200Mi
```

3. **Tune the liveness-probe timeouts** directly on the `dynatrace-oneagent-csi-driver` DaemonSet (`initialDelaySeconds: 15`, `periodSeconds: 15`, `timeoutSeconds: 10`, and `--probe-timeout=9s` on the liveness-probe container). Helm does not expose these — GitOps setups need a post-renderer.
4. **Disable the liveness probe** — last resort only; not recommended.

Alternatively, reduce simultaneous mounts by lowering max pods per node. If none of this resolves it, collect a support archive (Section 8) and open a support ticket.

3. OneAgent Issues

OneAgent Pods Not Starting

Diagnostic:

```
# Check DaemonSet
kubectl -n dynatrace get daemonset -l app.kubernetes.io/component=oneagent

# Check pods
kubectl -n dynatrace get pods -l app.kubernetes.io/component=oneagent -o wide

# Describe failing pod
kubectl -n dynatrace describe pod &lt;oneagent-pod-name&gt;
```

Common Causes:

Cause	Resolution
Tolerations missing	Add tolerations to DynaKube
Node selector mismatch	Update node selectors
Resource unavailable	Check node resources
Security context denied	Configure PSP/PSS

OneAgent Not Reporting

Diagnostic:

```
# Check OneAgent logs
kubectl -n dynatrace logs &lt;oneagent-pod&gt; -c oneagent --tail=100

# Check connectivity
kubectl -n dynatrace exec &lt;oneagent-pod&gt; -c oneagent -- curl -v
https://&lt;tenant&gt;.live.dynatrace.com/api/v1/deployment/installer/agent/co
nnectioninfo
```

Common Causes:

Cause	Resolution
Firewall blocking	Allow egress port 443
Proxy misconfiguration	Set proxy in DynaKube
Certificate issues	Check custom CA config

4. ActiveGate Issues

ActiveGate Not Starting

Diagnostic:

```
# Check StatefulSet
kubectl -n dynatrace get statefulset -l app.kubernetes.io/component=activegate

# Check pods
kubectl -n dynatrace get pods -l app.kubernetes.io/component=activegate

# Check logs
kubectl -n dynatrace logs &lt;activegate-pod&gt; --tail=100
```

Common Causes:

Cause	Resolution
Memory too low	Increase resources in DynaKube
PVC issues	Check storage class
Token invalid	Regenerate API token

No Kubernetes Metrics

Diagnostic:

```
# Check capabilities
kubectl -n dynatrace get dynakube -o
jsonpath='{.items[*].spec.activeGate.capabilities}'

# Should include: kubernetes-monitoring
```

Resolution: Ensure `kubernetes-monitoring` capability is enabled:

```
spec:
  activeGate:
    capabilities:
      - kubernetes-monitoring
      - routing
```

5. Injection Issues

Pods Not Getting Injected

Diagnostic:

```
# Check namespace labels
kubectl get namespace &lt;namespace&gt; -o jsonpath='{.metadata.labels}'

# Check pod annotations
kubectl get pod &lt;pod&gt; -o jsonpath='{.metadata.annotations}'

# Check webhook
kubectl get mutatingwebhookconfigurations
```

Common Causes:

Cause	Resolution
Namespace not selected	Add matching labels
Pod has opt-out annotation	Remove annotation
Webhook not registered	Restart operator
Code modules not delivered	With the CSI driver: check the `dynatrace-oneagent-csi-driver` pod on that node. With image volumes (Operator 1.11.0+): check the node meets the Kubernetes 1.35+ and runtime requirements. Otherwise check the init container's download (ephemeral storage)

Check Injection Status

```
# Check if init container present
kubectl get pod &lt;pod&gt; -o jsonpath='{.spec.initContainers[*].name}'

# Check environment variables
kubectl exec &lt;pod&gt; -c &lt;container&gt; -- env | grep DT_
```

Namespace Selector Debugging

The selector sits inside the injection mode, not at the top of `spec`:

```
# DynaKube selector (cloudNativeFullStack or applicationMonitoring)
kubectl -n dynatrace get dynakube -o
jsonpath='{.items[*].spec.oneAgent.cloudNativeFullStack.namespaceSelector}'
kubectl -n dynatrace get dynakube -o
jsonpath='{.items[*].spec.oneAgent.applicationMonitoring.namespaceSelector}'

# Label a namespace with whatever key/value that selector matches
kubectl label namespace &amp;lt;namespace&amp;gt; &amp;lt;key&amp;gt;=&amp;lt;value&amp;gt;
```

An empty result from both means no selector is set, and every namespace is in scope.

Documented Injection-Skip Reason Annotations

When the webhook skips injection it stamps the pod with `oneagent.dynatrace.com/injected: "false"` (likewise `metadata-enrichment.dynatrace.com/injected` and `otlp-exporter-configuration.dynatrace.com/injected`) plus a companion `reason` annotation. Read them with `kubectl describe pod <pod> -n <namespace>`:

Reason	Meaning	Typical Fix
`NoBootstrapperConfig`	Webhook can't find or create the bootstrapper config Secret in the pod's namespace	Incomplete DynaKube reconciliation — check DynaKube status and operator logs
`MissingTenantUUID`	DynaKube reconciliation incomplete; environment UUID not yet verified	Wait for / fix reconciliation (token, connectivity)
`DynaKubeStatusNotReady`	CodeModules status not ready — webhook can't determine which CodeModule to inject	Check DynaKube conditions; verify tenant connectivity
`OwnerLookupFailed`	Webhook can't determine the pod owner (name and kind)	Usually transient — Kubernetes API temporarily unreachable or slow; re-create the pod

Reason	Meaning	Typical Fix
`NoOTLPExporterConfigSecret`	Webhook can't find or create the OTLP exporter configuration Secret	Incomplete reconciliation or configuration preventing Secret creation
`IngestEndpointUnavailable`	Webhook can't construct a valid ingest endpoint URL at injection time	Check DynaKube `apiUrl` and tenant connectivity

6. Data Collection Issues

Missing Metrics

Diagnostic Steps:

1. Verify entity exists in Dynatrace
2. Check metric availability for entity type
3. Verify time range includes data
4. Check for filtering issues

```
// Verify Kubernetes entities are being discovered (smartscape topology)
smartscapeNodes "K8S_CLUSTER"
| fields entity.name = name, lifetime
| sort lifetime desc

// Legacy alternative (deprecated for new content):
// fetch dt.entity.kubernetes_cluster
// | fields entity.name, lifetime
// | sort lifetime desc
```

```
// Check if container metrics are flowing
timeseries cpu = avg(dt.kubernetes.container.cpu_usage), from:-1h
| limit 1
```

```
// Check if logs are being ingested
fetch logs, from: now() - 1h
| filter isNotNull(k8s.namespace.name)
| summarize logCount = count()
```

Missing Logs

Diagnostic:

```
# Is log monitoring configured on the DynaKube? (spec.logMonitoring)
kubectl -n dynatrace get dynakube -o jsonpath='{.items[*].spec.logMonitoring}'

# Verify OneAgent log collection
kubectl -n dynatrace exec &lt;oneagent-pod&gt; -c oneagent -- cat
/var/lib/dynatrace/oneagent/log/oneagent.log | tail -50
```

Common Causes:

Cause	Resolution
`spec.logMonitoring` not set	Add `spec.logMonitoring: {}` (K8S-07 § 3)
Application logs to a file, not stdout/stderr	The Log modules read stdout/stderr only — use a forwarder or custom log source
Log volume too high	Configure sampling/filtering
Log format unsupported	Configure custom parsing

7. Connectivity Issues

Testing Connectivity

```
# From OneAgent pod
kubectl -n dynatrace exec &lt;oneagent-pod&gt; -c oneagent -- \
  curl -v https://&lt;tenant&gt;.live.dynatrace.com/api/v1/time

# From ActiveGate pod
kubectl -n dynatrace exec &lt;activegate-pod&gt; -- \
  curl -v https://&lt;tenant&gt;.live.dynatrace.com/api/v1/time
```

Network Policy Issues

```
# Allow Dynatrace egress
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dynatrace-egress
  namespace: dynatrace
spec:
  podSelector: {}
  policyTypes:
    - Egress
  egress:
    - to:
        - ipBlock:
            cidr: 0.0.0.0/0
      ports:
        - protocol: TCP
          port: 443
```

Proxy Configuration

```
spec:
  proxy:
    value: https://proxy.example.com:8080
  # Or use a secret
  # proxy:
  #   valueFrom:
  #     secretKeyRef:
  #       name: proxy-secret
  #       key: proxy
```

8. Diagnostic Commands

Quick Health Check

```
#!/bin/bash
# dynatrace-health-check.sh

echo "=== DynaKube Status ==="
kubectl -n dynatrace get dynakube

echo "\n=== Dynatrace Pods ==="
kubectl -n dynatrace get pods

echo "\n=== OneAgent DaemonSet ==="
kubectl -n dynatrace get daemonset -l app.kubernetes.io/component=oneagent

echo "\n=== ActiveGate StatefulSet ==="
kubectl -n dynatrace get statefulset -l app.kubernetes.io/component=activegate

echo "\n=== Recent Events ==="
kubectl -n dynatrace get events --sort-by='.lastTimestamp' | tail -10
```

Collect a Support Archive (Official Subcommand)

The Operator ships a built-in `support-archive` subcommand that packages version info, logs from all Dynatrace components (OneAgent, ActiveGate, webhook, CSI driver), Kubernetes manifests for operator components and DynaKubes, and troubleshoot output into a single zip — this is what Dynatrace support asks for:

```
kubectl exec -n dynatrace deployment/dynatrace-operator -- \
  dynatrace-operator support-archive --stdout &gt; operator-support-
archive.zip
```

If the operator pod itself won't start, run the archive from a standalone pod using the operator's ServiceAccount and image:

```
kubectl run -n dynatrace support-archive --rm -i \
  --overrides='{ "spec": { "serviceAccount": "dynatrace-operator" } }' \
  --restart Never --image &lt;operator-image&gt; -- \
  support-archive --delay 10 --stdout &gt; support-archive.zip
```

Collect Support Bundle (Manual Fallback)

```
# Create support bundle
mkdir dynatrace-debug
cd dynatrace-debug

# DynaKube
kubectl -n dynatrace get dynakube -o yaml &gt; dynakube.yaml

# Pods
kubectl -n dynatrace get pods -o yaml &gt; pods.yaml

# Events
kubectl -n dynatrace get events &gt; events.txt

# Operator logs
kubectl -n dynatrace logs -l app.kubernetes.io/name=dynatrace-operator &gt;
operator.log

# OneAgent logs (one pod)
kubectl -n dynatrace logs $(kubectl -n dynatrace get pods -l
app.kubernetes.io/component=oneagent -o jsonpath='{.items[0].metadata.name}')
-c oneagent &gt; oneagent.log

# Package
cd ..
tar -czf dynatrace-debug.tar.gz dynatrace-debug/
```

Common kubectl Commands

Command	Purpose
`kubectl -n dynatrace get all`	All resources
`kubectl -n dynatrace describe dynakube`	DynaKube details
`kubectl -n dynatrace logs <pod>`	Pod logs
`kubectl -n dynatrace exec -it <pod> -- sh`	Shell access

9. DQL-Based Diagnostics

Complement kubectl diagnostics with DQL queries that detect Dynatrace component issues across your cluster fleet.

> **Query Kubernetes events on `event.provider`, not `event.kind`.** Kubernetes events land in the `events` object carrying `event.kind == "DAVIS_EVENT"` — the same kind as everything else Davis emits — so *kind* cannot discriminate them. The working discriminator is **`event.provider == "KUBERNETES_EVENT"`**, and the reason lives in **`dt.kubernetes.event.reason`** (`k8s.event.reason` resolves but is null on every record). A filter on `event.kind == "K8S_EVENT"` is syntactically valid, executes, and returns nothing — which during triage reads as "no failures," the most expensive possible wrong answer. Verified against a live tenant 08/11/2026.

Common Failure Patterns

Pattern	Symptoms	Detection Method
CSI Volume Timeout	Pods stuck in ContainerCreating	`dt.kubernetes.event.reason == "FailedMount"`
Injection Failure	Missing init container	`dt.kubernetes.event.reason` in `Failed` / `InspectFailed`, namespace `dynatrace`
ActiveGate scheduling failure	AG never starts, data gaps	`dt.kubernetes.event.reason == "FailedScheduling"`
OneAgent CrashLoop	Incomplete monitoring	`dt.kubernetes.event.reason == "BackOff"` (`CrashLoopBackOff` appears in `.message`)
OOM kill	AG/OA restarts, data gaps	`dt.kubernetes.container.oom_kills` metric — no reliable event reason
ActiveGate gone quiet	Stale data, no updates	`smartscapeNodes "ACTIVEGATE"` whose last-seen time is older than two hours (query below)

```
// Detect Dynatrace component failures in the last 24h
// One open event is re-written about every 5 min under the same event.id, so
it can repeat here.
// Discriminator is event.provider == "KUBERNETES_EVENT" — NOT event.kind,
which
// carries "DAVIS_EVENT" on these records and never takes a "K8S_EVENT" value.
// The reason field is dt.kubernetes.event.reason; k8s.event.reason is null
throughout.
fetch events, from:-24h
| filter event.provider == "KUBERNETES_EVENT"
| filter k8s.namespace.name == "dynatrace"
| filter in(dt.kubernetes.event.reason,
    {"Failed", "FailedMount", "FailedScheduling", "BackOff", "Killing",
"Unhealthy", "InspectFailed"})
| fields timestamp, k8s.cluster.name, k8s.workload.name,
         dt.kubernetes.event.reason,
         dt.kubernetes.event.involved_object.name,
         dt.kubernetes.event.message
| sort timestamp desc
| limit 50
```

```
// ActiveGates that stopped reporting — last seen more than 2 h ago
// Uses Smartscape node lifetimes (no bytes scanned). from:-7d is required:
without it only nodes
// seen in the default window are returned, so a long-silent ActiveGate is
invisible.
// A re-created ActiveGate pod gets a new node with the same name, so group by
name and take the
// latest last-seen time — a name with a current instance drops out.
// This replaces a dt.davis.events filter on "ACTIVEGATE" / "AGENT_CONNECTION"
event types, which
// matched nothing over 30 days on the validation tenant (10/02/2026): no such
event types exist there.
smartscapeNodes "ACTIVEGATE", from:-7d
| summarize {lastSeen = max(lifetime[end]), instances = count()}, by:{name}
| filter lastSeen &lt; now() - 2h
| sort lastSeen desc
```

```
// Hosts with gaps in CPU reporting (OneAgent interruptions)
// arraySize counts empty buckets too, so count only the non-null ones.
timeseries hostCpu = avg(dt.host.cpu.usage), from:-6h, by:{dt.entity.host}
| fieldsAdd buckets = arraySize(hostCpu), reported =
arraySize(arrayRemoveNulls(hostCpu))
| fieldsAdd completeness = round(100.0 * reported / buckets, decimals: 1)
| filter completeness &lt; 95.0
| fieldsAdd hostName = entityName(dt.entity.host, type:"dt.entity.host")
| fields hostName, completeness, reported, buckets
| sort completeness asc
// A host that stopped reporting entirely drops out of timeseries altogether —
// cross-check smartscapeNodes "HOST" for hosts with no series.
```

10. Symptom-to-Resolution Index

The Dynatrace community maintains a curated Kubernetes/OpenShift troubleshooting map (Dynatrace community) — a living index of exact error messages mapped to resolution articles, kept current by Dynatrace staff. The tables below index the most load-bearing entries by the error text you will actually see; use the map itself for the full, growing list. (A few entries live on the *Heads-up from Dynatrace* board, which requires community sign-in.)

Deployment and Startup

Error / Symptom	Resolution
`CrashLoopBackOff: Downgrading OneAgent is not supported, please uninstall the old version first`	Downgrading OneAgent (Dynatrace community)
Crash loop on pods when installing OneAgent	Crash loop on install (Dynatrace community)
Deployment succeeded but `dynatrace-oneagent` container not ready / not running / no meaningful logs / image can't be pulled	"Deployment seems successful, but…" family (Dynatrace community) — sibling articles cover each variant

Error / Symptom	Resolution
`ImagePullBackOff` on OneAgent / ActiveGate pods	ImagePullBackoff (Dynatrace community); on Kubernetes 1.35+ with a private registry, *"the kubelet verifies pull credentials per pod, even for images already cached on the node"*, so injected pods without their own pull secret fail with `ImagePullBackOff` — see Use a private registry (DT docs) and ONBRD-05 §5
`CannotPullContainerError`	CannotPullContainerError (Dynatrace community)
No Dynatrace pods scheduled on control-plane nodes	Control-plane taints/tolerations (Dynatrace community)
Error applying the DynaKube custom resource on GKE	GKE custom resource error (Dynatrace community)
Pods stuck in `Terminating` after an upgrade	Pods stuck terminating (Dynatrace community)
OneAgent / CSI pod count doesn't match node count	OA/CSI pod count mismatch (Dynatrace community)

Connectivity and Authentication

Error / Symptom	Resolution
`There was an error with the TLS handshake`	TLS handshake (Dynatrace community)
`Invalid bearer token`	Invalid bearer token (Dynatrace community)
Problem with ActiveGate token	ActiveGate token (Dynatrace community)
`Internal error occurred: failed calling webhook (…) x509: certificate signed by unknown authority`	Webhook x509 certificate (Dynatrace community)
`Could not check credentials. Process is started by other user`	Credentials / process user (Dynatrace community)
OneAgent unable to connect when Istio is enabled	OneAgent + Istio connectivity (Dynatrace community)

Error / Symptom	Resolution
Dynatrace Kubernetes service missing / creation fails with Istio enabled	Istio service creation (Dynatrace community)
Connectivity issues with Calico network policies	Calico connectivity (Dynatrace community)

Injection

Error / Symptom	Resolution
Cloud Native Full-Stack pods not injected — systematic validation walkthrough	CNFS pod injection validation (Dynatrace community)
Application pods can't start post-injection	Pods can't start post-injection (Dynatrace community)
OpenShift: injected pods rejected at admission (seccomp × SCC)	FAQ-13 — Dynatrace injection and OpenShift SCCs
`MountVolume` error migrating Classic Full-Stack → Cloud Native Full-Stack	CFS→CNFS MountVolume error (Dynatrace community)
Injection skipped with a `reason` annotation	Section 5 above — documented reason codes

Data Gaps and Metrics

Error / Symptom	Resolution
Deployment successful but no monitoring data in Dynatrace	No monitoring data (Dynatrace community)
Gaps in cluster monitoring after changing Kubernetes settings	Gaps after settings change (Dynatrace community)
Node memory usage reported greater than memory limits	Node memory vs limits (Dynatrace community)
PVC metrics not showing data	PVC not showing data (Dynatrace community)

Error / Symptom	Resolution
"Monitoring not available" problem on an inactive cluster	Inactive cluster (Dynatrace community)
Which dashboards / alerts / SLOs still use deprecated Kubernetes metrics?	Deprecated K8s metrics audit (Dynatrace community)

Prometheus Annotation Scraping

Error / Symptom	Resolution
Excluded Prometheus metrics still fetched by ActiveGate	Excluded metrics still fetched (Dynatrace community)
Prometheus metrics missing	Missing Prometheus metrics (Dynatrace community)
`Counter metric cache limit exceeded` warning in ActiveGate logs	Counter metric cache limit (Dynatrace community)

Summary

In this notebook, you learned:

- Systematic troubleshooting methodology
- Operator debugging and common issues
- OneAgent troubleshooting steps
- ActiveGate diagnostics
- Code injection debugging
- Data collection issue resolution
- Connectivity testing and network policy configuration
- Diagnostic commands and the official Operator support-archive subcommand
- A symptom-to-resolution index built on the community troubleshooting map

References

- Operator + DynaKube troubleshooting (DT docs)
- Set up Dynatrace on Kubernetes (DT docs)
- How K8s monitoring works (DT docs)
- DynaKube parameters (DT docs)
- Davis Problems app (DT docs)
- Kubernetes/OpenShift troubleshooting map (Dynatrace community)
- Dynatrace Operator releases (Dynatrace GitHub)
- Operator 1.10.2 release notes (DT docs)
- Kubernetes Operator CSI driver crashes frequently (Dynatrace community)
- Cloud Native Full-Stack pod injection validation (Dynatrace community)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.